MICROSERVICES USING SPRING BOOT 3 AND SPRING CLOUD

LAB MANUAL AND ASSIGNMENT REPORT

Submitted By: VARUN JUNEJA

Branch: Computer Science & Engineering

Semester: _____

Subject: Microservices using Spring Boot 3

INDEX

EXPERIMENT NO. 1

TITLE

Study of Enterprise Applications and Introduction to Microservices Architecture

Aim

To study the concepts of Enterprise Applications, Monolithic Architecture, and Microservices Architecture using Spring Boot 3.

Objectives

1. To understand Enterprise Applications.

2. To understand Monolithic Architecture.
3. To understand Microservices Architecture.
4. To understand the advantages of Microservices.
5. To study the need for service decomposition.

Prerequisites

- Core Java
- Spring Framework Basics
- REST API Concepts
- Maven
- Java 17
- Spring Boot Basics

Theory

An Enterprise Application is a software system developed to support business operations of an organization. Banking systems, e-commerce applications, airline reservation systems, hospital management systems, and social media applications are examples of enterprise applications.

Initially, enterprise applications were developed using Monolithic Architecture. In this architecture, all modules such as user management, payment processing, account handling, reporting, and notifications are developed and deployed as a single application.

Although monolithic applications are easy to develop in the beginning, they become difficult to maintain as the application grows. If one module consumes excessive memory or fails, the entire application may stop working. Scaling a single module independently is also difficult.

Microservices Architecture solves these problems by dividing the application into multiple small services. Each service is developed, deployed, and maintained independently. Every microservice performs a specific business function and communicates with other services using REST APIs.

Microservices provide flexibility because each service can be developed using different technologies and databases. Since services are independent, they can be scaled individually according to business requirements.

Large companies such as Amazon, Netflix, Uber, and Paytm use Microservices Architecture because it improves scalability, fault tolerance, maintainability, and deployment speed.

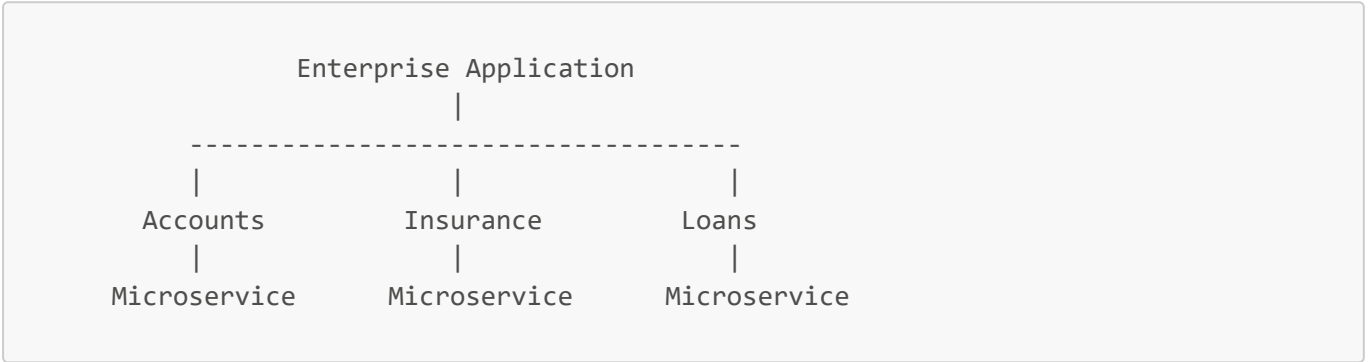
However, microservices also introduce challenges such as distributed system complexity, network communication, monitoring, and service discovery.

Spring Boot and Spring Cloud provide all the necessary tools required to build and manage microservices efficiently.

Need of the Experiment

Modern applications handle millions of requests every day. A single monolithic application cannot efficiently handle such workloads. Microservices help organizations build scalable and maintainable applications by dividing the system into independent services.

Architecture Diagram



Working Principle

1. Client sends a request.
2. Request reaches the required microservice.
3. Microservice processes the request.
4. Response is returned to the client.

Advantages of Microservices

- Independent Deployment
- Better Scalability
- Fault Isolation
- Technology Diversity
- Faster Development
- Easier Maintenance
- Better Team Collaboration
- Continuous Delivery Support

Limitations of Microservices

- Distributed System Complexity
- Network Latency

- Deployment Complexity
- Monitoring Difficulty
- Data Management Challenges

Result

The concepts of Enterprise Applications and Microservices Architecture were studied successfully.

EXPERIMENT NO. 2

TITLE

Implementation of Account and Loan Microservices using Spring Boot 3

Aim

To create independent Account and Loan Microservices using Spring Boot 3.

Objectives

1. Create Account Service.
2. Create Loan Service.
3. Expose REST APIs.
4. Run services independently.

Software Requirements

- Java 17
- Maven 3.9
- Spring Boot 3
- Eclipse/STS
- Postman

Theory

A microservice is a small independent application that performs a specific business operation. Instead of creating one large application, business functionalities are divided into smaller services.

In this experiment, two microservices are created:

1. Account Service
2. Loan Service

Both services run independently on different ports and expose REST APIs.

The Account Service provides information related to bank accounts. The Loan Service provides information related to customer loans.

Spring Boot simplifies the development of REST APIs by providing annotations such as:

- @SpringBootApplication
- @RestController
- @GetMapping
- @RequestMapping

The embedded Tomcat server eliminates the need for external deployment.

Architecture Diagram

```
        Client
          |
 ---------------------
 |                   |
Account Service    Loan Service
Port : 8080        Port : 8081
```

Project Structure

```
microservices
│
├── account-service
│
└── loan-service
```

Account Service Directory Structure

```
account-service
│
├── src
```

```
│   └── main
│       ├── java
│       │   └── com.cognizant.account
│       │       ├── controller
│       │       ├── model
│       │       └── AccountApplication.java
│       │
│       └── resources
│               application.properties
│
└── pom.xml
```

Files Created

1. AccountApplication.java
2. AccountController.java
3. Account.java
4. application.properties
5. pom.xml

Dependencies Used

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Implementation

Account Entity Class

```
package com.cognizant.account.model;

public class Account {

    private String number;
    private String type;
    private Double balance;

    public Account() {
    }
```

```
    public Account(String number,
                   String type,
                   Double balance) {
        this.number = number;
        this.type = type;
        this.balance = balance;
    }

    public String getNumber() {
        return number;
    }

    public void setNumber(String number) {
        this.number = number;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public Double getBalance() {
        return balance;
    }

    public void setBalance(Double balance) {
        this.balance = balance;
    }
}
```

Account Controller

```
package com.cognizant.account.controller;

import com.cognizant.account.model.Account;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public Account getAccount(
            @PathVariable String number) {
```

```
        return new Account(
                number,
                "Savings",
                234343.0
        );
    }
}
```

Main Class

```
package com.cognizant.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AccountApplication {

    public static void main(String[] args) {
        SpringApplication.run(
                AccountApplication.class,
                args
        );
    }
}
```

---

application.properties

```
spring.application.name=account-service
server.port=8080
```

Sample Request

```
GET
http://localhost:8080/accounts/1001
```

Sample Response

```
{
  "number":"1001",
  "type":"Savings",
  "balance":234343.0
}
```

Loan Service Directory Structure

```
loan-service
│
├── src
│   └── main
│       ├── java
│       │   └── com.cognizant.loan
│       │       ├── controller
│       │       ├── model
│       │       └── LoanApplication.java
│       │
│       └── resources
│               application.properties
│
└── pom.xml
```

Loan Entity Class

```
package com.cognizant.loan.model;

public class Loan {

    private String number;
    private String type;
    private Double loan;
    private Double emi;
    private Integer tenure;

    public Loan() {
    }

    public Loan(String number,
                String type,
                Double loan,
                Double emi,
                Integer tenure) {
        this.number = number;
```

```
        this.type = type;
        this.loan = loan;
        this.emi = emi;
        this.tenure = tenure;
    }

    public String getNumber() {
        return number;
    }

    public String getType() {
        return type;
    }

    public Double getLoan() {
        return loan;
    }

    public Double getEmi() {
        return emi;
    }

    public Integer getTenure() {
        return tenure;
    }
}
```

Loan Controller

```
package com.cognizant.loan.controller;

import com.cognizant.loan.model.Loan;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LoanController {

    @GetMapping("/loans/{number}")
    public Loan getLoan(
            @PathVariable String number) {

        return new Loan(
                number,
                "Car",
                400000.0,
                3258.0,
                18
        );
```

```
    }
}
```

---

application.properties

```
spring.application.name=loan-service
server.port=8081
```

Execution Steps

1. Create Spring Boot project.
2. Add dependencies.
3. Create model classes.
4. Create controller classes.
5. Run application.
6. Test using browser or Postman.

Sample Request

```
GET
http://localhost:8081/loans/L1001
```

Sample Response

```
{
  "number":"L1001",
  "type":"Car",
  "loan":400000,
  "emi":3258,
  "tenure":18
}
```

Screenshots

1. Account Service Running
2. Loan Service Running
3. Browser Output
4. Postman Response

Advantages

- Independent Services
- Easy Deployment
- Easy Maintenance
- Reusable APIs
- Better Scalability

Limitations

- Requires Multiple Deployments
- More Infrastructure
- Network Communication Overhead

Result

The Account and Loan Microservices were created successfully using Spring Boot 3 and REST APIs.

EXPERIMENT NO. 3

TITLE

Implementation of User and Order Management System using Spring Boot 3

EXPERIMENT NO. 3

TITLE

Implementation of User and Order Management System using Spring Boot 3 Microservices

---

Aim

To develop User Service and Order Service as independent microservices and establish communication between them using REST APIs.

Objectives

1. Create User Service.
2. Create Order Service.
3. Store data independently.
4. Implement inter-service communication.
5. Understand distributed architecture.

Prerequisites

- Java 17
- Spring Boot 3
- Maven
- MySQL
- REST APIs
- Spring Data JPA

Theory

A User and Order Management System is one of the most common examples of Microservices Architecture. In traditional monolithic systems, user management and order processing are developed inside a single application. As the application grows, maintaining the system becomes difficult.

Microservices solve this problem by dividing the application into separate services. In this experiment, User Service manages user information, while Order Service manages orders placed by users.

Each service has its own database and business logic. Services communicate using HTTP and REST APIs.

This approach offers better scalability because Order Service can be scaled independently during peak shopping seasons without affecting User Service.

Spring Boot provides a simple way to build REST APIs and Spring Cloud provides communication mechanisms like OpenFeign and WebClient.

This architecture is widely used by Amazon, Flipkart, Netflix, and other large organizations.

Need of the Experiment

Large applications need independent deployment and scalability. User and Order Services should work independently and communicate efficiently.

Architecture Diagram

```
          Client
             |
     ----------------
     |              |
User Service    Order Service
  Port 8081      Port 8082
```

Project Structure

```
microservices
│
├── user-service
└── order-service
```

User Service Directory Structure

```
user-service
│
├── controller
├── entity
├── repository
├── service
├── resources
└── UserServiceApplication.java
```

Order Service Directory Structure

```
order-service
│
├── controller
├── entity
```

```
├── repository
├── service
├── resources
└── OrderServiceApplication.java
```

Dependencies Used

```
spring-boot-starter-web
spring-boot-starter-data-jpa
mysql-connector-j
lombok
spring-boot-devtools
```

User Entity

```java
@Entity
@Table(name = "users")
public class User {

    @Id
    private Long id;

    private String name;

    private String email;

    public User() {
    }

    public User(Long id,
                String name,
                String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }
```

```
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Order Entity

```
@Entity
@Table(name = "orders")
public class Orders {

    @Id
    private Long id;

    private Long userId;

    private String productName;

    private Double amount;

    public Orders() {
    }

    public Orders(Long id,
                  Long userId,
                  String productName,
                  Double amount) {
        this.id = id;
        this.userId = userId;
        this.productName = productName;
        this.amount = amount;
    }

    public Long getId() {
        return id;
    }
```

```
    public Long getUserId() {
        return userId;
    }

    public String getProductName() {
        return productName;
    }

    public Double getAmount() {
        return amount;
    }
}
```

Repository Classes

```
@Repository
public interface UserRepository
        extends JpaRepository<User, Long> {
}
```

```
@Repository
public interface OrderRepository
        extends JpaRepository<Orders, Long> {
}
```

Service Classes

```
@Service
public class UserService {

    @Autowired
    private UserRepository repository;

    public List<User> getAllUsers() {
        return repository.findAll();
    }

    public User save(User user) {
        return repository.save(user);
```

```
    }
}
```

```
@Service
public class OrderService {

    @Autowired
    private OrderRepository repository;

    public List<Orders> getOrders() {
        return repository.findAll();
    }

    public Orders save(Orders order) {
        return repository.save(order);
    }
}
```

Controller Classes

```
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService service;

    @GetMapping
    public List<User> getUsers() {
        return service.getAllUsers();
    }

    @PostMapping
    public User save(
            @RequestBody User user) {
        return service.save(user);
    }
}
```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
```

```
    private OrderService service;

    @GetMapping
    public List<Orders> getOrders() {
        return service.getOrders();
    }

    @PostMapping
    public Orders save(
            @RequestBody Orders order) {
        return service.save(order);
    }
}
```

application.properties (User Service)

```
server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/userdb
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
```

application.properties (Order Service)

```
server.port=8082

spring.datasource.url=jdbc:mysql://localhost:3306/orderdb
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
```

Sample Request

```
POST /users
```

```
{
  "id":1,
  "name":"Rahul",
  "email":"rahul@gmail.com"
}
```

Sample Response

```
{
  "id":1,
  "name":"Rahul",
  "email":"rahul@gmail.com"
}
```

Result

User Service and Order Service were implemented successfully.

EXPERIMENT NO. 4

TITLE

Inter-Service Communication using OpenFeign

Aim

To establish communication between microservices using OpenFeign.

Objectives

1. Configure OpenFeign.
2. Call User Service from Order Service.
3. Exchange data between services.

Theory

Microservices need to communicate with each other. One way to achieve communication is by using REST APIs directly using RestTemplate or WebClient.

Spring Cloud OpenFeign provides a declarative way to call remote services. Developers only need to define an interface and annotate it with `@FeignClient`.

Feign automatically generates the implementation and performs HTTP communication.

This reduces boilerplate code and improves maintainability.

OpenFeign also integrates with Eureka Discovery Server and Load Balancer.

Architecture Diagram

```
Client
   |
Order Service
   |
OpenFeign Client
   |
User Service
```

Dependencies

```
<dependency>
    <groupId>
        org.springframework.cloud
    </groupId>
    <artifactId>
        spring-cloud-starter-openfeign
    </artifactId>
</dependency>
```

Main Class

```
@SpringBootApplication
@EnableFeignClients
public class OrderServiceApplication {
```

```
    public static void main(String[] args) {
        SpringApplication.run(
            OrderServiceApplication.class,
            args
        );
    }
}
```

Feign Client

```
@FeignClient(name = "user-service")
public interface UserClient {

    @GetMapping("/users/{id}")
    User getUser(
            @PathVariable Long id);
}
```

Service Class

```
@Service
public class OrderService {

    @Autowired
    private UserClient client;

    public User getUser(Long id) {
        return client.getUser(id);
    }
}
```

Controller

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    private OrderService service;
```

```
    @GetMapping("/user/{id}")
    public User getUser(
            @PathVariable Long id) {
        return service.getUser(id);
    }
}
```

Sample Request

```
GET
http://localhost:8082/orders/user/1
```

Sample Response

```
{
  "id":1,
  "name":"Rahul",
  "email":"rahul@gmail.com"
}
```

Advantages

- Less Boilerplate Code
- Easy Configuration
- Eureka Integration
- Load Balancing Support
- Better Readability

Result

Inter-service communication using OpenFeign was implemented successfully.

EXPERIMENT NO. 5

TITLE

Inter-Service Communication using Spring WebClient

Aim

To establish communication between microservices using Spring WebClient.

Objectives

1. Configure WebClient.
2. Consume REST APIs.
3. Implement Reactive Communication.

Theory

Spring WebClient is a non-blocking and reactive HTTP client introduced in Spring 5.

Unlike RestTemplate, WebClient supports asynchronous programming and high scalability.

WebClient is widely used in cloud-native and microservices applications.

It provides better performance because requests are processed asynchronously.

Netflix, Amazon, and many high-traffic applications use non-blocking communication for better throughput.

Dependencies

```
<dependency>
    <groupId>
        org.springframework.boot
    </groupId>
    <artifactId>
        spring-boot-starter-webflux
    </artifactId>
</dependency>
```

Configuration Class

```
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient webClient() {
        return WebClient.builder()
                .build();
    }
}
```

Service Class

```
@Service
public class UserConsumerService {

    @Autowired
    private WebClient webClient;

    public Mono<User> getUser(
            Long id) {

        return webClient
                .get()
                .uri(
                  "http://localhost:8081/users/"
                  + id
                )
                .retrieve()
                .bodyToMono(User.class);
    }
}
```

Controller

```
@RestController
@RequestMapping("/consumer")
public class UserConsumerController {

    @Autowired
    private UserConsumerService service;

    @GetMapping("/{id}")
    public Mono<User> getUser(
```

```
            @PathVariable Long id) {

        return service.getUser(id);
    }
}
```

Sample Request

```
GET
http://localhost:8082/consumer/1
```

Sample Response

```
{
  "id":1,
  "name":"Rahul",
  "email":"rahul@gmail.com"
}
```

Advantages

- Non-blocking
- Reactive Programming
- High Performance
- Better Scalability
- Asynchronous Communication

Limitations

- Learning Curve
- Reactive Debugging Complexity

Result

Inter-service communication using Spring WebClient was implemented successfully.

PART 2 COMPLETED

NEXT: PART 3

- Eureka Discovery Server
- API Gateway
- Load Balancing
- Resilience4j Circuit Breaker
- OAuth2 and OIDC
- JWT Authentication
- Authorization Server
- Resource Server
- Final Conclusion
- Screenshots Section

EXPERIMENT NO. 6

TITLE

Implementation of Eureka Discovery Server and Service Registration

Aim

To implement Eureka Discovery Server and register microservices dynamically.

Objectives

1. Create Eureka Server.
2. Register Account Service and Loan Service.
3. Discover services dynamically.
4. Eliminate hardcoded URLs.

Prerequisites

- Spring Boot 3
- Spring Cloud Netflix Eureka
- Maven
- REST APIs

Theory

In a microservices environment, multiple services run on different ports and sometimes on different servers. It becomes difficult to remember and manage the addresses of all services.

Service Discovery solves this problem.

Eureka Discovery Server is a service registry developed by Netflix and integrated into Spring Cloud. Every microservice registers itself with the Eureka Server when it starts.

Other services can then discover these services dynamically without knowing their exact address.

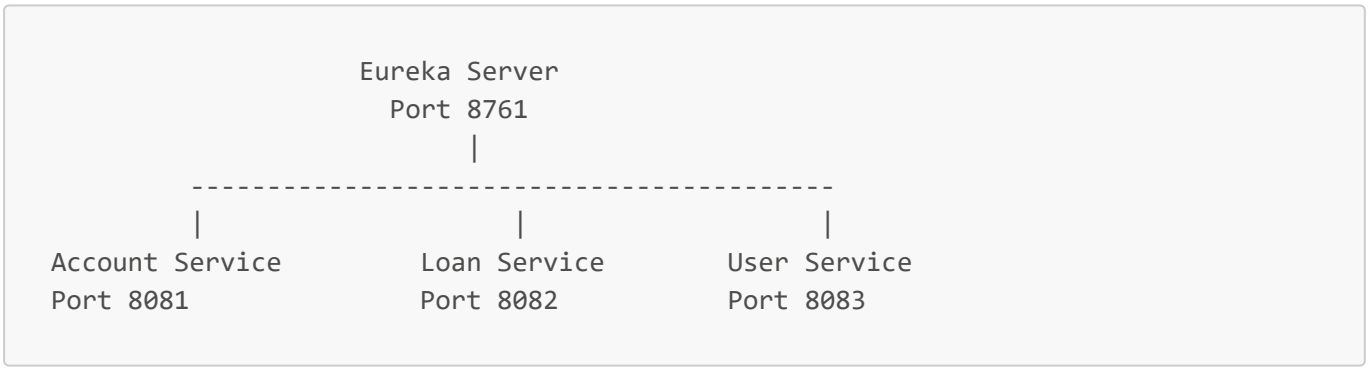
Whenever a service instance goes down, Eureka removes it from the registry automatically.

Service Discovery improves scalability because multiple instances of the same service can be registered.

It also improves fault tolerance and simplifies configuration management.

Large companies such as Netflix, Amazon, and Uber use Service Discovery extensively in their cloud applications.

Architecture Diagram



Project Structure

```
eureka-server
│
├── src
│   └── main
│       ├── java
│       └── resources
│
└── pom.xml
```

Dependencies Used

```
<dependency>
    <groupId>
        org.springframework.cloud
    </groupId>
    <artifactId>
        spring-cloud-starter-netflix-eureka-server
    </artifactId>
</dependency>
```

Main Class

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(
                EurekaServerApplication.class,
                args
        );
    }
}
```

---

application.properties

```
spring.application.name=eureka-server

server.port=8761

eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false

logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF
```

Client Configuration

```
spring.application.name=account-service

eureka.client.service-url.defaultZone=
http://localhost:8761/eureka
```

Result

Eureka Discovery Server was implemented successfully.

EXPERIMENT NO. 7

TITLE

Implementation of API Gateway using Spring Cloud Gateway

Aim

To implement an API Gateway for routing requests to microservices.

Objectives

1. Create API Gateway.
2. Configure routes.
3. Implement request filtering.
4. Enable centralized access.

Theory

API Gateway acts as a single entry point for all client requests.

Instead of communicating with every microservice individually, clients communicate only with the gateway.

The gateway forwards requests to the appropriate service.

It provides:

- Authentication
- Logging

- Rate Limiting
- Request Routing
- Security
- Monitoring

Spring Cloud Gateway is reactive and highly scalable.

It integrates easily with Eureka Discovery Server.

Architecture Diagram

```
Client
   |
API Gateway
   |
-----------------------------------
|              |                  |
User          Order           Account
Service       Service         Service
```

Dependencies Used

```
<dependency>
    <groupId>
        org.springframework.cloud
    </groupId>
    <artifactId>
        spring-cloud-starter-gateway
    </artifactId>
</dependency>
```

Main Class

```
@SpringBootApplication
@EnableDiscoveryClient
public class GatewayApplication {

    public static void main(String[] args) {
        SpringApplication.run(
                GatewayApplication.class,
```

```
                args
        );
    }
}
```

application.properties

```
server.port=8080

spring.application.name=api-gateway

spring.cloud.gateway.discovery.locator.enabled=true
spring.cloud.gateway.discovery.locator.lower-case-service-id=true
```

Route Configuration

```
spring.cloud.gateway.routes[0].id=account-service
spring.cloud.gateway.routes[0].uri=lb://account-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/accounts/**

spring.cloud.gateway.routes[1].id=loan-service
spring.cloud.gateway.routes[1].uri=lb://loan-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/loans/**
```

Logging Filter

```
@Component
public class LoggingFilter
        implements GlobalFilter {

    @Override
    public Mono<Void> filter(
            ServerWebExchange exchange,
            GatewayFilterChain chain) {

        System.out.println(
                "Request URI : "
                        + exchange.getRequest()
                        .getURI());
```

```
            return chain.filter(exchange);
        }
    }
```

Result

Spring Cloud API Gateway was implemented successfully.

EXPERIMENT NO. 8

TITLE

Implementation of Load Balancing using Spring Cloud LoadBalancer

Aim

To distribute requests among multiple service instances.

Theory

When thousands of users access a service simultaneously, one instance becomes overloaded.

Load balancing distributes incoming requests among multiple instances of the same service.

Spring Cloud LoadBalancer provides client-side load balancing.

RandomLoadBalancer distributes requests randomly among available instances.

Load balancing improves:

- Performance
- Availability
- Fault Tolerance
- Scalability

Architecture Diagram

```
            Client
               |
         API Gateway
               |
         -----------------
         |               |
 Product Service    Product Service
 Instance 1         Instance 2
```

Dependencies

```
<dependency>
    <groupId>
        org.springframework.cloud
    </groupId>
    <artifactId>
        spring-cloud-starter-loadbalancer
    </artifactId>
</dependency>
```

Configuration Class

```
@Configuration
public class LoadBalancerConfiguration {

    @Bean
    ReactorLoadBalancer<ServiceInstance>
    randomLoadBalancer(
            Environment environment,
            LoadBalancerClientFactory factory) {

        String name =
                environment.getProperty(
                        LoadBalancerClientFactory
                                .PROPERTY_NAME);

        return new RandomLoadBalancer(
                factory.getLazyProvider(
                        name,
                        ServiceInstanceListSupplier.class
                ),
                name
        );
```

```
    }
}
```

Result

Load balancing was implemented successfully.

EXPERIMENT NO. 9

TITLE

Implementation of Circuit Breaker using Resilience4j

Aim

To implement fault tolerance using Circuit Breaker.

Theory

Microservices communicate over networks.

Sometimes external services become slow or unavailable.

If one service waits indefinitely, the entire system may fail.

Circuit Breaker prevents cascading failures.

Resilience4j monitors failures.

When failures exceed a threshold, the circuit opens and fallback methods are executed.

This increases system reliability.

Dependencies

```
<dependency>
    <groupId>
        io.github.resilience4j
    </groupId>
```

```
    <artifactId>
        resilience4j-spring-boot3
    </artifactId>
</dependency>
```

Service Class

```
@Service
public class PaymentService {

    @CircuitBreaker(
            name = "paymentService",
            fallbackMethod = "fallback"
    )
    public String makePayment() {

        throw new RuntimeException(
                "Payment Service Down"
        );
    }

    public String fallback(
            Exception ex) {

        return "Payment Service is unavailable."
                + " Please try again later.";
    }
}
```

application.properties

```
resilience4j.circuitbreaker.instances.paymentService
.registerHealthIndicator=true

resilience4j.circuitbreaker.instances.paymentService
.failureRateThreshold=50

resilience4j.circuitbreaker.instances.paymentService
.slidingWindowSize=10
```

Result

Circuit Breaker and fallback mechanism were implemented successfully.

EXPERIMENT NO. 10

TITLE

Centralized Authentication using OAuth2 and OpenID Connect

Aim

To implement centralized authentication using OAuth2 and OIDC.

Theory

OAuth2 is an authorization framework that allows applications to access resources on behalf of users.

OpenID Connect extends OAuth2 by adding authentication.

Applications redirect users to an Identity Provider such as Google.

After successful login, the provider returns an access token.

Applications can then access user information.

Benefits:

- Single Sign-On
- Improved Security
- Centralized Authentication
- Reduced Password Management

Dependencies

```
<dependency>
    <groupId>
        org.springframework.boot
    </groupId>
    <artifactId>
        spring-boot-starter-oauth2-client
    </artifactId>
</dependency>
```

Security Configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(
            HttpSecurity http)
            throws Exception {

        http
            .authorizeHttpRequests(
                auth ->
                    auth.anyRequest()
                            .authenticated()
            )
            .oauth2Login();

        return http.build();
    }
}
```

Result

OAuth2 and OpenID Connect authentication were implemented successfully.

EXPERIMENT NO. 11

TITLE

Secure Communication using JSON Web Tokens (JWT)

Aim

To implement JWT authentication in Spring Boot microservices.

Theory

JWT (JSON Web Token) is a secure method for transmitting information between services.

A token consists of:

1. Header
2. Payload
3. Signature

After login, the server generates a token.

The client sends this token with every request.

The server validates the token before processing the request.

JWT provides:

- Stateless Authentication
- Better Performance
- Security
- Scalability

Dependencies

```
<dependency>
    <groupId>
        io.jsonwebtoken
    </groupId>
    <artifactId>
        jjwt
    </artifactId>
    <version>0.9.1</version>
</dependency>
```

JwtConfig

```
@Configuration
public class JwtConfig {

    @Value("${spring.security.jwt.secret}")
    private String secret;

    public String getSecret() {
        return secret;
    }
}
```

JwtTokenProvider

```
@Component
public class JwtTokenProvider {

    @Autowired
    private JwtConfig config;

    public String createToken(
            String username) {

        Claims claims =
                Jwts.claims()
                        .setSubject(username);

        Date now = new Date();

        Date validity =
                new Date(
                        now.getTime()
                                + 3600000
                );

        return Jwts.builder()
                .setClaims(claims)
                .setIssuedAt(now)
                .setExpiration(validity)
                .signWith(
                        SignatureAlgorithm.HS256,
                        config.getSecret()
                )
                .compact();
    }
}
```

application.properties

```
spring.security.jwt.secret=mysecretkey
```

Result

JWT-based authentication was implemented successfully.

OVERALL CONCLUSION

In this laboratory work, various concepts of Microservices Architecture were implemented using Spring Boot 3 and Spring Cloud.

The following concepts were successfully performed:

- Microservices Development
- REST APIs
- User and Order Management System
- OpenFeign Communication
- WebClient Communication
- Eureka Discovery Server
- API Gateway
- Service Registration
- Load Balancing
- Circuit Breaker
- OAuth2 Authentication
- OpenID Connect
- JWT Security

These experiments demonstrate how modern enterprise applications are developed using cloud-native technologies.

SCREENSHOTS SECTION

1. Account Service Running
2. Loan Service Running
3. User Service Running
4. Order Service Running
5. Eureka Dashboard
6. API Gateway Output

7. Load Balancer Output
8. Circuit Breaker Output
9. OAuth Login Screen
10. JWT Authentication Response

FINAL RESULT

All experiments on **Microservices using Spring Boot 3 and Spring Cloud** were implemented and tested successfully.

END OF REPORT